

Carroyage-JMT

Documentation technique
développeurs & maintenance

Version 22.12

Mise à jour 2026-05-05

Dépôt github.com/BaDjams/Carroyage-JMT

Table des matières

Table des matières

Table des matières

1. Vue d'ensemble

2. Stack technique

3. Architecture générale

- 3.1 Structure SPA monolithique
- 3.2 Ordre de chargement des scripts
- 3.3 État global (variables window)
- 3.4 Contrats inter-modules via le DOM

4. Modes opérationnels

- Mode 1 — Carroyage rapide (CADO)
- Mode 2 — Export de zone
- Mode 3 — Créer MBTiles

5. Référence des fichiers

5.1 Scripts métier

- index.html (~150 KB)
- utilities.js
- version.js

5.2 Systèmes de carroyage

- carroyageCado.js (~43 KB)
- carroyageCFSI.js (~14 KB)
- carroyageUTM.js (~19 KB)

5.3 Modules d'export

- carroyageToCSV.js (~7.5 KB)
- carroyageToMbtiles.js (~16 KB)
- imagetoprint.js (~28 KB)

5.4 Mode 3 — création MBTiles

- mbtilesCreator.js (~32 KB)
- seedManager.js (~8.6 KB)
- tileSource.js

5.5 Orchestrateurs et helpers

- zoneDownloader.js
- settingsManager.js

5.6 Couches cartographiques

- map-layers.js

5.7 Bibliothèques d'icônes

- icons.js
- icons-catalog.js (~213 KB, généré)

build_library.py

5.8 PWA

manifest.json

sw.js — Service Worker

6. Systèmes de carroyage

6.1 CADO (système maison)

6.2 UTM

6.3 CFSI / DFCI

7. Couches cartographiques (map-layers.js)

7.1 Structure

7.2 Couches disponibles

7.3 Ajout d'une couche

7.4 HERE Maps (différé)

8. Pipelines d'export

8.1 KML / KMZ

8.2 GeoJSON / GPX

8.3 CSV (WKT)

8.4 PNG haute résolution

8.5 MBTiles (drone DJI)

8.6 Seeds

9. Street View & DualMaps

9.1 Architecture

9.2 Capture d'écran

9.3 Iframe DualMaps en mode sombre

10. Service Worker / PWA

10.1 Stratégie de cache

10.2 Exceptions de cache

10.3 Proxy CORS Yandex

10.4 Cycle de vie

10.5 Procédure de release

11. Configuration & secrets

11.1 config.private.js (gitignored)

11.2 Chargement gracieux

11.3 Important — historique purgé

12. Build & déploiement

12.1 Image Docker

12.2 docker-compose.yml

12.3 Lancer en local (développement)

13. CI/CD GitLab

13.1 Pipeline (.gitlab-ci.yml)

Stage build : docker-build

Stage deploy : docker-deploy

13.2 Conséquence pour les développeurs

13.3 Branches

14. Conventions de code et patterns

14.1 Style JS

14.2 CSS

14.3 Patterns récurrents

14.4 Convention de commit

15. Maintenance & évolutions

15.1 Tâches récurrentes

15.2 Pour ajouter un mode

15.3 Pour ajouter un format d'export

15.4 Pour ajouter un système de carroyage

15.5 Évolutions souhaitables

16. Troubleshooting

16.1 Le Service Worker ne met pas à jour

16.2 Tuiles Yandex en noir

16.3 Couche privée invisible

16.4 Export MBTiles tronqué / message "canvas trop grand"

16.5 Force-push GitLab refusé

16.6 Conflit après push CI

16.7 Aperçu Leaflet figé

16.8 KMZ exporté sans icônes

16.9 Tailwind z-[10002] n'a pas d'effet

Annexe A — Glossaire

Annexe B — Liens utiles

Carroyage-JMT — Documentation technique

Version applicative : 22.12 **Public visé** : développeurs et mainteneurs du projet **Date de mise à jour** : 2026-05-05 **Dépôt** : <https://github.com/BaDjams/Carroyage-JMT> — miroir GitLab : <https://gitlab.promogi.fr/romain.jamet/Carroyage-JMT>

Table des matières

1. Vue d'ensemble
 2. Stack technique
 3. Architecture générale
 4. Modes opérationnels
 5. Référence des fichiers
 6. Systèmes de carroyage
 7. Couches cartographiques
 8. Pipelines d'export
 9. Street View & DualMaps
 10. Service Worker / PWA
 11. Configuration & secrets
 12. Build & déploiement
 13. CI/CD GitLab
 14. Conventions de code et patterns
 15. Maintenance & évolutions
 16. Troubleshooting
-

1. Vue d'ensemble

Carroyage-JMT (alias **CADO**) est une application web monopage (PWA) écrite en HTML/CSS/JavaScript pur, sans étape de bundling. Elle permet de générer des carroyages tactiques (grilles de référence) sur fond cartographique selon trois systèmes de coordonnées (CADO, UTM, CFSI/DHCI), de les exporter dans plusieurs formats (KML, KMZ, GeoJSON, GPX, CSV, MBTiles, PNG haute résolution), de prévisualiser en temps réel sur carte Leaflet et de fonctionner hors-ligne grâce à un service worker.

Domaine d'usage : opérations de terrain (sécurité civile, gendarmerie, drones DJI, recherches en zones étendues).

URL de production : <https://cado.promogi.fr> (déploiement Docker derrière nginx-proxy).

2. Stack technique

Couche	Technologie
Front-end	HTML5, CSS3, JavaScript ES2017+ vanilla (pas de framework)
UI	Tailwind CSS (précompilé) + Flowbite + style.css
Cartographie	Leaflet 1.x + Leaflet.draw
SQLite navigateur	sql.js (WASM) — <code>sql-wasm.js</code> + <code>sql-wasm.wasm</code>
Compression	JSZip + FileSaver.js
Encodage géo	Open Location Code (Plus Codes)
PWA	Service Worker natif + manifest.json
Stockage local	localStorage (icônes POI) + OPFS (Origin Private File System) pour gros MBTiles
Build (icônes)	Python 3 + Pillow
Conteneurisation	nginx:alpine-slim (Docker)
CI/CD	GitLab CI (<code>docker:dind</code> + auto-deploy via push de tag)

Aucun gestionnaire de paquets JS (pas de `package.json`). Toutes les libs tierces sont vendorées dans le repo.

3. Architecture générale

3.1 Structure SPA monolithique

`index.html` (~150 KB) contient toute l'UI dans un unique document. Les trois modes opérationnels sont des `<div>` cachés/affichés via JS :

```
#cado-mode-container    → Mode 1 (Carroyage rapide CAD0)
#zone-mode-container    → Mode 2 (Export de zone)
#creator-mode-container → Mode 3 (Créer MBTiles)
```

Le bouton de bascule porte un attribut `data-mode` lu par le routeur JS dans `index.html`.

3.2 Ordre de chargement des scripts

L'ordre dans `index.html` est important — plusieurs fichiers exposent des variables globales utilisées par les suivants :

1. version.js → APP_VERSION
2. JSZip / FileSaver → libs tierces
3. utilities.js → fonctions partagées (rotation, conversions)
4. map-layers.js → MAP_LAYERS, clés API
5. icons.js / icons-catalog.js → ICON_LIBRARY, ICON_CATALOG
6. carroyageCado.js → générateur CADO + init Leaflet mode 1
7. carroyageCFSI.js → CFSI_UTILS
8. carroyageUTM.js → WGS84_to_UTM
9. zoneDownloader.js → orchestrateur mode 2
10. carroyageToMbtiles.js / carroyageToCSV.js → exports
11. seedManager.js → encodage seeds
12. tileSource.js → abstraction MBTiles/online
13. mbtilesCreator.js → mode 3
14. settingsManager.js → gestion icônes utilisateur
15. imagetoprint.js → export PNG haute résolution

3.3 État global (variables `window`)

Variable	Rôle
<code>window.cadoMap</code> , <code>window.zoneMap</code> , <code>window.creatorMap</code>	instances Leaflet par mode
<code>window.userPOIs</code>	tableau des points d'intérêt utilisateur
<code>loadedCadoKmlFeatures</code> , <code>loadedZoneKmlFeatures</code>	features KML importées
<code>cadoKmlResources</code> , <code>kmlResources</code>	images embarquées dans KMZ importés
<code>currentIconLibrary</code>	bibliothèque d'icônes courante (modifiable)
<code>MAP_LAYERS</code> , <code>ICON_LIBRARY</code> , <code>ICON_CATALOG</code>	catalogues statiques
<code>CFSI_UTILS</code> , <code>WGS84_to_UTM</code>	namespaces de conversion

3.4 Contrats inter-modules via le DOM

Plutôt que des imports/exports, les modules communiquent via des IDs DOM stables. Modifier ces IDs casse le couplage :

ID / classe	Lecteur principal	Écrivain principal
<code>#decimal-coords</code> , <code>#dms-coords</code> , <code>#utm-coords</code> , <code>#mercator-coords</code>	carroyageCado.js	UI utilisateur
<code>#cado-interactive-map</code>	carroyageCado.js (init)	—
<code>#zone-interactive-map</code>	zoneDownloader.js	—
<code>#creator-interactive-map</code>	mbtilesCreator.js	—
<code>.grid-parameter</code> (classe)	carroyageCado.js (debounce 400ms)	UI
<code>.color-option[data-color][data-name]</code>	carroyageCado.js, zoneDownloader.js	—
<code>#settings-modal</code>	settingsManager.js	—
<code>.mbtiles-status-badge</code> , <code>.mbtiles-info-bar</code>	tileSource.js	—
<code>#cado-mbtiles-input</code> , <code>#zone-mbtiles-input</code>	tileSource.js	—

4. Modes opérationnels

Mode 1 — Carroyage rapide (CADO)

Génère un quadrillage CADO centré sur un point unique (lat/lon en décimal, DMS, UTM ou Mercator). Aperçu temps réel sur carte Leaflet, exports KML/KMZ/GeoJSON/GPX/CSV/PNG/MBTiles.

Fichier pivot : `carroyageCado.js` (~43 KB).

Mode 2 — Export de zone

L'utilisateur dessine un rectangle (Leaflet.draw) ou importe un KML/KMZ existant. Génère plusieurs grilles superposées (UTM + CFSI + CADO, au choix) sur la zone, gère les POI utilisateur.

Fichier pivot : `zoneDownloader.js` .

Mode 3 — Créer MBTiles

Interface dédiée pour générer une base MBTiles (fond de carte tuilé hors-ligne) sur une zone et plage de zoom choisies. Utilise OPFS pour stocker > 100 000 tuiles sans saturer la RAM.

Fichier pivot : `mbtilesCreator.js` + `seedManager.js` .

5. Référence des fichiers

5.1 Scripts métier

`index.html` (~150 KB)

Shell SPA. Contient : - la balise `<head>` avec les imports CSS - les trois conteneurs `*-mode-container` cachés - les modales (paramètres, aide, Street View, DualMaps) - les boutons de mode et le routeur JS qui bascule l'affichage - la logique Street View / DualMaps (clic droit → menu contextuel) — voir §9

Maintenance : ce fichier est gros et monolithique. Toute modification structurelle nécessite une recherche sur `id=` ou `data-*` avant édition pour vérifier le couplage avec les autres scripts.

`utilities.js`

Fonctions transverses : - **Numérotation cellules** : `letterToNumber(c)`, `numberToLetter(n)` — supporte indices négatifs (Z, Y, ..., A, B, ..., Y, Z) avec un saut au-dessus de 0 - **Indices de grille** : `generateIndices(start, end)` (saute 0), `getOffsetInCells(n)`, `getNextIndex(n)` - **Géométrie cœur** : `calculateAndRotatePoint(colNumber, rowNumber, config, a1Lat, a1Lon)` — convertit coords cellule → lat/lon avec correction cosinus de latitude et rotation via matrice autour d'un pivot. La correction utilise `config.latitude` (centre) et `non a1Lat` pour éviter une asymétrie en bord de zone - **Rendu canvas** : `drawLabelWithOutline()`, `drawSubdivisionKey()` — étiquettes avec contour et barre d'échelle - **UI** : `downloadFile()`, `showError()`, `hideError()`

Toutes les rotations passent par cette fonction. Modifier les axes ou la convention demande beaucoup de précautions : les exports KML, l'aperçu Leaflet et le PNG haute résolution doivent rester cohérents.

`version.js`

```
const APP_VERSION = '22.12';
```

Source unique de la version. Le `CACHE_NAME` du Service Worker (`cado-cache-vXX`) est en revanche manuellement synchronisé.

5.2 Systèmes de carroyage

`carroyageCado.js` (~43 KB)

Module le plus important. Génère le quadrillage CADO (cellules à origine A1, taille fixe).

Fonctions principales : - `generateGrid()` — orchestrateur d'export (lit l'UI, dispatche selon format choisi) - `getGridConfiguration(lat, lon)` — extrait config depuis l'UI - `calculateGridData(config)` — géométrie : lignes, points, rotations, avec `cache` pour éviter les recomputations - `updateCadoGridPreview()` — rendu Leaflet temps réel, `debounce 400 ms` via `schedulePreviewUpdate`

- `generateKML()` , `generateKMZ()` — exports avec génération inline d'icônes (lettres dessinées sur canvas pour KMZ) - `generateGeoJSON()` , `generateGPX()` - `handleCadoKmzFile()` — import KML/KMZ via JSZip, extrait images embarquées dans `cadoKmlResources.images`

Algorithmes notables : - **Lettrage** : supporte sens ascendant/descendant, indices négatifs, double saisie (étiquettes en haut et à droite) - **Choix de référence** : pivot à l'origine A1 ou au centre — recalcule symétriquement les coins - **Rotation** : appliquée comme matrice finale après projection Mercator

`carroyageCFSI.js` (~14 KB)

Système CFSI/DFCI français (Lambert 93 / NTF, mailles 100 m).

Exports : - `CFSI_UTILS` (IIFE) avec : conversions WGS84 ↔ Lambert II-E, parsing de codes - `drawCfsiGridOnCanvas(ctx, bbox, latLonToPixels, margin, fontSize, lineWidth)` — rasterise grille + étiquettes - `drawCfsiCartouche()` — cartouche titre/précision

Algorithmes : - Helmert WGS84 → NTF : `DX=168, DY=60, DZ=-320` - Lambert II-E : 6 itérations de raffinement de latitude (précision sub-métrique) - Décodage : Lambert → carré 100 km (alphabet 13×20) → 20 km → 2 km → 100 m - **Rendu adaptatif** selon zoom : pleins codes (< 700 m), 100 m avec coloration (< 3500 m), 2 km au-delà

`carroyageUTM.js` (~19 KB)

Système UTM (Universal Transverse Mercator), mailles 1 km, multi-zones.

Exports : - `WGS84_to_UTM` (IIFE) avec `fromLatLon(lat, lon, forceZone)` , `toLatLon(...)` , `getUTMZoneLetter(lat)` - `generateUTMGrid()` — export KMZ avec POI - `calculateGridForZoneStrip(...)` — génère lignes par zone avec clipping - `createUTM_KML()` — organise lignes par zone et type (Easting/Northing)

Algorithmes : - **Clipping multi-zones** : pour chaque zone dans la plage, lignes tronquées via Cohen-Sutherland (`clipLineToRect()` , `clipAxis()`) - Limites de zones : tous les 6° de longitude, avec exceptions Norvège/Svalbard (zones 31V, 32V, 31X, 33X, 35X, 37X) - False easting : 500 000 m ; false northing : 10 000 000 m (hémisphère sud)

5.3 Modules d'export

`carroyageToCSV.js` (~7.5 KB)

Export WKT (compatible Google Earth Pro, QGIS, ArcGIS).

```
generateGridCSV(filename, useUtm, useCfsi, useCado, userPOIs, optionalCadoData)
```

- Mode 1 : reçoit `optionalCadoData = {config, gridData}` directement
- Mode 2 : lit l'UI via `getZoneCadoConfigAndBounds()`
- Sortie : WKT `LINESTRING` (lignes) + `POINT` (étiquettes/POIs)

`carroyageToMbtiles.js` (~16 KB)

Génère MBTiles (SQLite + tuiles PNG) pour drones DJI, zoom 17-19.

Fonctions clés : - `generateMbtilesProcess(filename, useUtm, useCfsi, useCado, bbox, baseZoom, userPOIs, optionalCadoData)` - `processZoomLevel(...)` — pour chaque niveau de zoom : trace tout sur canvas global puis découpe - `sliceAndStore(db, sourceCanvas, zoom, globalNwPx, globalSePx)` — découpe canvas en tuiles 256×256, insertion SQL

Conventions : - TMS : Y inversé (`tmsY = (1 << z) - 1 - y`) - Limite : `maxCanvasSize = 8192 px` — alerte au-delà - Pré-cache des images POI avant la boucle de zoom - Fonctions de rendu **digitales** (sans halo) : `drawDigitalUtm()`, `drawDigitalCfsiStrict()`, `drawDigitalCado()`, `drawDigitalKml()`, `drawDigitalPois()`

`imagetoprint.js` (~28 KB)

Export PNG haute résolution (impression).

- `generateImageToPrint()` — télécharge tuiles, dessine grille CADO, exporte PNG
- En mode "centre", recalcule un BBox **symétrique** autour du centre de la grille (gère les colonnes négatives correctement)
- `iterativeFetchTiles()` — fetch quadtree depth-first
- `drawSubdivisionKey()` — barre d'échelle, flèche nord, sous-grille tous les 50 px
- Deux chemins de tuiles : MBTiles (`tileSourceIsActive()`) ou provider en ligne

5.4 Mode 3 — création MBTiles

`mbtilesCreator.js` (~32 KB)

Interface Mode 3 (carte Leaflet + contrôles + barre de progression).

- `initCreatorMode()` — initialise la carte avec toutes les couches disponibles
- `updateCreatorUI()` — barre de progression + alertes (seuil **8000 tuiles** RAM, **100 000** avec OPFS)
- `checkOPFS()` — détecte le support du stockage privé navigateur
- `seedTiles(selectedBbox, selectedZoom)` — orchestre via `seedManager.js`

Spécifique : gestion projection EPSG:3395 pour Yandex (correction nécessaire), gestion QuadKey pour Bing.

`seedManager.js` (~8.6 KB)

Encodage compact (base64, 11 caractères) d'une configuration de carroyage pour partage par URL.

Layout 8 octets : | Bits | Champ | Plage | |---|---|---| | 0–20 | latInt | 21 bits, ~11 m de précision | | 21–42 | lonInt | 22 bits | | 43–52 | scale/10 | 10 bits, max 10 230 m | | 53–55 | type de grille | 3 bits | | 56–59 | flags | swap axes, double entrée, sens lettrage, point de réf |

`tileSource.js`

Abstraction unifiée pour les tuiles : MBTiles vs en ligne.

- `tileSourceLoad(file)` — parse MBTiles (sql.js), extrait métadonnées + niveaux de zoom
- `tileSourceReadTile(x, y, z)` — query SQL → blob URL
- `tileSourceIsActive()`, `tileSourceGetZooms()`, `tileSourceGetBestZoom(targetZoom)`

Cache la base SQLite en variable de module `_tsDB`. Met à jour `.mbtiles-status-badge` et masque le sélecteur de provider en ligne quand un MBTiles est actif.

5.5 Orchestrateurs et helpers

zoneDownloader.js

Orchestrateur Mode 2 + utilitaires de tuiles partagés.

- `handleZoneVectorExport()` — lit la zone, dispatche selon checkboxes (UTM/CFSI/CADO/KML/CSV/MBTILES)
- `generateZonePNG()` — rasterisation PNG
- POI : `addPointMode()`, `removePoint()`, `getUserPOIs()`
- KML : `handleZoneKmlFile()`, parsing
- Icônes : `getIconsSync()`, cache `cachedIconLibrary`
- Projections Yandex : `_yMerc3395()`, `_inverseMerc3395()`, `_latLonToTile3395()`

settingsManager.js

Gestion bibliothèque d'icônes utilisateur (modal #settings-modal).

- `initSettingsManager()` — init modal + persistance `localStorage['userIcons']`
- `renderSettingsTable()`, `saveSettingsFromTable()`
- Import/export JSON + drag-drop

5.6 Couches cartographiques

map-layers.js

Catalogue statique. Voir §7 pour la structure détaillée.

Pattern de clés privées : 1. `config.private.js` (gitignored) déclare `var IGN_PRIVATE_API_KEY`, `var MAPY_API_KEY`, `var GOOGLE_MAPS_API_KEY` 2. `map-layers.js` a des **fallbacks** `if (typeof X === 'undefined') var X = ''`; 3. Les couches avec `"requiresKey": "VAR_NAME"` sont filtrées dans `createBaseLayers()` si la variable est vide

Pourquoi `var` et non `const` : `var` autorise la redéclaration entre `config.private.js` et `map-layers.js`. Avec `const`, redéclaration = erreur fatale.

5.7 Bibliothèques d'icônes

icons.js

Bibliothèque par défaut compacte (`ICON_LIBRARY` : 7 entrées : start, end, danger, info, camp, camera, star).

icons-catalog.js (~213 KB, généré)

Catalogue professionnel (ADVERSAIRE, JUDICIAIRE, etc.) généré automatiquement par `build_library.py`. Ne pas éditer à la main.

Format :

```
ICON_CATALOG = [{ id, label, path: [...], category, url, scale, order }, ...]
```

`build_library.py`

Script Python (Pillow). Workflow : 1. Parcourt `icons/sticker/` récursivement 2. Optimise les PNG → `icons/sticks/` (64×64, RGBA) 3. Index chaque icône avec métadonnées 4. Écrit `icons-catalog.js`

Exécution : `python build_library.py` après ajout/modification d'icônes.

5.8 PWA

`manifest.json`

```
{
  "start_url": "./index.html",
  "display": "standalone",
  "theme_color": "#3b82f6",
  "icons": [192×192, 512×512]
}
```

`sw.js` — Service Worker

Voir §10 pour la stratégie complète.

6. Systèmes de carroyage

6.1 CADO (système maison)

- **Origine** : cellule A1 placée par l'utilisateur (mode "origine") ou autour d'un point central (mode "centre")
- **Taille de maille** : configurable (défaut 10 m)
- **Numérotation** : lettres en X (A, B, ..., Z, AA, AB, ...), nombres en Y. Sens et axes interchangeables
- **Indices négatifs** : possibles, on saute 0 (... , -B, -A, A, B, ...)
- **Rotation** : libre, en degrés
- **Sortie** : KML, KMZ, GeoJSON, GPX, CSV, PNG, MBTiles

6.2 UTM

- **Maille fixe** : 1 km
- **Zones** : 6° de longitude, gestion multi-zones (clipping aux limites)
- **Étiquettes** : easting/northing en kilomètres
- **Exceptions** : Norvège (32V), Svalbard (31X, 33X, 35X, 37X)

6.3 CFSI / DFCI

- **Maille** : 100 m, nichée dans des carrés 2 km, 20 km, 100 km
 - **Projection** : Lambert II étendu (NTF Helmert)
 - **Code** : ex. **KH18A2** (carré 100 km + 20 km + 2 km + 100 m)
 - **Usage typique** : sécurité civile, lutte incendies forêt
-

7. Couches cartographiques (**map-layers.js**)

7.1 Structure

```
{
  id: "google_hybrid",           // identifiant interne
  name: "Google Hybrid",        // libellé UI
  maxZoom: 21,
  requiresKey: "GOOGLE_MAPS_API_KEY", // optionnel
  layers: [                     // empilement de couches
    { url: "...", type: "xyz" | "quadkey" | "yandex" }
  ]
}
```

7.2 Couches disponibles

ID	Nom	Source	maxZoom	Clé requise
<code>ign_ign_hybrid</code>	Ortho IGN + Routes IGN	data.geopf.fr (public)	19	non
<code>ign_google_hybrid</code>	Ortho IGN + Routes Google	IGN + Google	19	non
<code>yandex_hybrid</code>	Yandex Hybride (FR)	maps.yandex.net	18	non
<code>bing_hybrid</code>	Bing Maps Hybride	virtualearth.net	19	non
<code>google_hybrid</code>	Google Hybrid	mt0.google.com	21	non
<code>google_hybrid_NOPOI</code>	Google Hybrid sans POI	mt0.google.com (apistyle)	21	non
<code>ign_scan_composite</code>	IGN Cartes (privé)	data.geopf.fr/private	18	<code>IGN_PRIVATE_API_KEY</code>
<code>ign_public_hybrid</code>	Plan IGN	data.geopf.fr	19	non
<code>mapy_outdoor</code>	Mapy.CZ Outdoor	api.mapy.com	19	<code>MAPY_API_KEY</code>
<code>osm_standard</code>	OpenStreetMap	tile.openstreetmap.org	19	non

7.3 Ajout d'une couche

1. Ajouter une entrée à `MAP_LAYERS` dans `map-layers.js`
2. Si privée : ajouter un fallback `if (typeof X === 'undefined') var X = '';` en haut du fichier, et `requiresKey: "X"` dans l'entrée
3. Documenter la clé dans `config.private.js` (template à demander au mainteneur)

7.4 HERE Maps (différé)

Tâche en attente : ajout des couches HERE Maps dès qu'une clé sera obtenue sur platform.here.com.

```
// Ajout futur :  
// var HERE_API_KEY = '';  
// Carto : https://maps.hereapi.com/v3/base/mc/{z}/{x}/{y}/png8?style=explore.day&apiKey=${HERE_A  
// Hybrid : https://maps.hereapi.com/v3/base/mc/{z}/{x}/{y}/png8?style=explore.satellite.day&apiK
```

8. Pipelines d'export

Toutes les exports passent par le même `gridData` (lignes + étiquettes) calculé une seule fois, puis dispatché.

8.1 KML / KMZ

- KML : XML simple
- KMZ : zip d'un KML + dossier `images/` (icônes des étiquettes générées sur canvas, lettres en gros)
- Préserve les sources KML importées (re-embed via JSZip)

8.2 GeoJSON / GPX

- GeoJSON : `FeatureCollection` de `LineString` + `Point`
- GPX : `<rte>` + `<wpt>` (compatible randonnée, navigateurs GPS)

8.3 CSV (WKT)

Format compatible QGIS/Google Earth Pro : une colonne `WKT` + colonnes label/type/etc.

8.4 PNG haute résolution

- Récupère les tuiles dans le BBox cible (online ou MBTiles)
- Trace la grille par-dessus
- Ajoute légende (échelle, nord)
- Adapte la largeur de trait pour upscaling lisible

8.5 MBTiles (drone DJI)

- SQLite 3 (sql.js) en mémoire
- Tuiles PNG 256×256 transparentes (overlay)
- Convention TMS (Y inversé)
- Plage de zoom typique : 17-19
- Limite canvas : 8192 px (alerte au-delà)

8.6 Seeds

- Encodage 8 octets → 11 caractères base64
- Partage par URL : `?seed=<11chars>`

9. Street View & DualMaps

Ajouté récemment dans `index.html` . Clic droit sur la carte → menu contextuel : - **Voir le Street View ici** : modal 360° via Google Maps JS API (si `GOOGLE_MAPS_API_KEY` présente) ou fallback iframe `output=svembed` - **Voir sur DualMaps** : iframe vers `data.mapchannels.com/dualmaps8/`

9.1 Architecture

- Backdrop + modal en z-index 10001/10002 (inline `style="z-index:..."`, pas Tailwind `z-[...]` qui ne fonctionne pas en précompilé)
- Boutons overlay (X et 📷) : centrés en haut, déplacés dynamiquement dans `document.fullscreenElement` lors du `fullscreenchange` natif de Google
- Mode JS API : `StreetViewService.getPanorama()` + `StreetViewPanorama` + minimap Google liée (`map.setStreetView(panorama)` → flèche bleue tournant en live)
- Mode iframe : pas de minimap, capture impossible

9.2 Capture d'écran

`getDisplayMedia` + crop via `getBoundingClientRect()` sur canvas. Boutons overlay masqués pendant la capture (2 rAF avant snapshot).

9.3 Iframe DualMaps en mode sombre

Astuce : `color-scheme: light; background: white` sur l'iframe → force un fond blanc, contourne le rendu noir-sur-noir des liens.

10. Service Worker / PWA

10.1 Stratégie de cache

`sw.js` : - `CACHE_NAME` : `cado-cache-v36` — à incrémenter manuellement à chaque release qui change la liste d'assets ou un fichier critique - `ASSETS_TO_CACHE` : liste exhaustive des fichiers à pré-cacher à l'install - Stratégie par défaut : `cache-first, falling back to network`

10.2 Exceptions de cache

Le SW ignore (laisse le réseau gérer) les requêtes vers : - `google.com`, `geopf.fr`, `openstreetmap`, `virtualearth` (tuiles cartographiques — sinon explosion du cache) - `data:`, `blob:`

10.3 Proxy CORS Yandex

Cas spécial : les tuiles Yandex sont relayées par le SW pour ajouter `Access-Control-Allow-Origin: *`. Sans ça, `canvas.drawImage()` les marque comme `tainted` et bloque les exports PNG/MBTiles.

```
if (url.includes('maps.yandex.net')) {  
  // fetch + ré-emballage avec headers CORS permissifs  
}
```

10.4 Cycle de vie

- `install` : `skipWaiting()` + `cache.addAll(ASSETS_TO_CACHE)` (n'échoue pas si un fichier non critique manque)

- `activate` : supprime les anciens caches, `clients.claim()` → contrôle immédiat des onglets ouverts

10.5 Procédure de release

À chaque release modifiant le code : 1. Bumper `APP_VERSION` dans `version.js` 2. Bumper `CACHE_NAME` dans `sw.js` (`cado-cache-vXX` → `vXX+1`) 3. Ajouter d'éventuels nouveaux fichiers à `ASSETS_TO_CACHE` 4. Commit + push → CI déclenche le build Docker

Oublier le bump de cache = utilisateurs bloqués sur l'ancienne version (PWA installée).

11. Configuration & secrets

11.1 `config.private.js` (gitignored)

Fichier non commité (cf. `.gitignore`). Template type :

```
var IGN_PRIVATE_API_KEY = 'xxx';
var MAPY_API_KEY = 'xxx';
var GOOGLE_MAPS_API_KEY = 'xxx';
```

11.2 Chargement gracieux

```
<script src="config.private.js" onerror="console.warn('config.private.js absent')"></script>
```

Si le fichier est absent (ex: clone fraîche), l'app fonctionne en mode dégradé : les couches privées sont masquées, Street View bascule en iframe.

11.3 Important — historique purgé

Le fichier `config.private.example.js` a été purgé de l'historique git (filter-branch sur 733 commits, force-push sur GitHub et GitLab) car il contenait des clés réelles à un moment donné. Les clés concernées ont été ou doivent être régénérées : - IGN privée : `50qNaab...` - Mapy : `68gdL0cv...`

Ne jamais commiter de clés, même dans un fichier `.example` . Utiliser uniquement `config.private.js` (gitignored) en local.

12. Build & déploiement

12.1 Image Docker

```
FROM nginx:alpine-slim
COPY . /usr/share/nginx/html
```

Image ultra-simple : nginx sert les fichiers statiques tels quels.

12.2 docker-compose.yml

```
services:
  carroyage:
    image: docker.promogi.fr/romain.jamet/carroyage-jmt:<COMMIT_SHA>
    restart: always

  proxy:
    image: docker.promogi.fr/neogi/infra/nginx-proxy:20260718.3
    restart: always
    ports: [443:443]
    environment:
      NGINX_SERVER_NAME: cado.promogi.fr
      NGINX_PROXY: http://carroyage:80
```

Le tag d'image est mis à jour automatiquement par la CI à chaque build sur `main` (commit `[skip ci] Deploy image XXX`).

12.3 Lancer en local (développement)

Pas de build : ouvrir `index.html` derrière un serveur statique (le Service Worker exige HTTP, pas `file://`) :

```
# Option 1 : Python
python -m http.server 8080

# Option 2 : npx
npx serve .

# Option 3 : Docker local
docker build -t carroyage-local . && docker run -p 8080:80 carroyage-local
```

Puis ouvrir `http://localhost:8080`.

En dev local, mettre les clés dans `config.private.js` à la racine (déjà gitignored).

13. CI/CD GitLab

13.1 Pipeline (`.gitlab-ci.yml`)

Deux stages, déclenchés **uniquement** sur `main` :

Stage `build` : `docker-build`

1. Build de l'image avec tag `<commit-short-sha>`
2. Push de l'image vers le registry GitLab (`registry.gitlab.com/...`)
3. Push aussi le tag `:latest`

Stage `deploy` : `docker-deploy`

1. Image alpine, installe `git`
2. Checkout du commit, hard-reset
3. `sed` : remplace dans `docker-compose.yml` l'ancien tag d'image par le nouveau `${CI_COMMIT_SHORT_SHA}`
4. Commit `[skip ci] Deploy image XXX` et push sur la branche d'origine

13.2 Conséquence pour les développeurs

Chaque push sur `main` génère **deux commits** dans l'historique GitLab : - celui du dev - celui de la CI (`[skip ci] Deploy image ...`)

Pour aligner GitHub et GitLab après une modif locale : 1. Push sur GitHub 2. Pull la branche depuis GitLab pour récupérer le commit `[skip ci]` de la CI 3. Re-push sur GitHub si besoin

Ou plus simple : **commits de dev sur GitHub, déploiement via push miroir sur GitLab.**

13.3 Branches

Depuis le nettoyage du 2026-05-05, **seule `main` existe sur GitLab**. Les anciennes branches obsolètes (`dev` , `test-GPT` , `merge-github-into-gitlab` , `CarroyageCADO_PWA` , `CarroyageCADO_PWA_Dev`) ont été supprimées.

`main` est protégée sur GitLab. Pour un force-push exceptionnel, il faut **déprotéger temporairement** la branche dans Settings → Repository → Protected Branches.

14. Conventions de code et patterns

14.1 Style JS

- ES2017+ vanilla, pas de transpilation
- IIFE pour les namespaces (`CFSI_UTILS` , `WGS84_to_UTM`)
- Globales `window.xxx` assumées (pas de modules ES)
- Commentaires en français

14.2 CSS

- Tailwind précompilé : les valeurs arbitraires `z-[10002]` , `bg-[#abc]` ne fonctionnent pas car le CSS est généré en build statique. Utiliser `style="z-index:10002"` inline
- `style.css` pour les overrides custom

14.3 Patterns récurrents

- Debounce 400 ms sur les changements de paramètres pour le preview Leaflet
- Cache de calcul : `gridData` mémoisé tant que la config ne change pas
- Canvas full-world puis crop pour le rendu MBTiles (au lieu de calcul tuile par tuile)
- `requiresKey` : pattern de feature-flag pour les couches privées
- TMS Y inversé : convention systématique pour MBTiles

14.4 Convention de commit

Pas de Conventional Commits stricts, mais préfixes courants : - `[skip ci] Deploy image XXX` — automatique CI - `Update <fichier>` — modifs simples - Messages descriptifs en français pour les features

15. Maintenance & évolutions

15.1 Tâches récurrentes

Tâche	Fréquence	Procédure
Bump version + cache SW	À chaque release	\$10.5
Mise à jour libs tierces	Trimestriel	Remplacer les <code>.min.js</code> vendorés et tester
Régénération <code>icons-catalog.js</code>	Sur ajout d'icônes	<code>python build_library.py</code>
Audit des clés API	Semestriel	Vérifier quotas, régénérer si fuite

15.2 Pour ajouter un mode

1. Créer un conteneur `<div id="newmode-mode-container">` caché dans `index.html`
2. Ajouter le bouton de mode avec `data-mode="newmode"`
3. Créer le module JS dédié (suivre l'archi `mbtilesCreator.js`)
4. Ajouter le script à `ASSETS_TO_CACHE` dans `sw.js` + bumper le cache

15.3 Pour ajouter un format d'export

1. Créer un fichier `carroyageToXxx.js` exposant `generateGridXxx(...)`
2. L'inscrire dans le dispatcher de `carroyageCado.js` (Mode 1) et/ou `zoneDownloader.js` (Mode 2)
3. Ajouter une option dans les radio buttons de l'UI (`zone-file-format` ou équivalent)

4. Cacher (`sw.js`) + bumper

15.4 Pour ajouter un système de carroyage

1. Créer `carroyageXxx.js` exposant les fonctions de calcul + rendu canvas
2. Ajouter les options UI (checkbox dans Mode 2)
3. Étendre les modules d'export (CSV, MBTiles, KML) pour le supporter

15.5 Évolutions souhaitables

- Découper `index.html` : actuellement monolithique. Une approche template (HTML imports natifs ou simple concat de fragments) faciliterait la maintenance
 - Tests : aucun test automatisé actuellement. Un harnais Playwright sur les exports core (KML, MBTiles) renforcerait la régression
 - Migration Tailwind JIT : permettrait les classes arbitraires (`z-[xxxx]`)
 - Modules ES natifs : remplacer les globales `window.xxx` par `import/export` quand on quittera la compatibilité totale (PWA installée)
-

16. Troubleshooting

16.1 Le Service Worker ne met pas à jour

Symptôme : modifs invisibles après push, même après refresh. **Cause** : `CACHE_NAME` non bumpé. **Solution** : incrémenter `cado-cache-vXX` dans `sw.js`, redéployer. Côté utilisateur : DevTools → Application → Service Workers → Unregister + hard refresh.

16.2 Tuiles Yandex en noir

Cause : SW pas activé (HTTP local sans hot-reload du SW), donc pas de proxy CORS. **Solution** : recharger après activation du SW, ou désactiver Yandex en dev local.

16.3 Couche privée invisible

Cause : `config.private.js` absent ou clé vide. **Solution** : créer le fichier à la racine ou utiliser une autre couche.

16.4 Export MBTiles tronqué / message "canvas trop grand"

Cause : zone × zoom dépasse 8192 px de canvas. **Solution** : réduire la zone, baisser le zoom max, ou (à terme) implémenter un rendu par sous-régions.

16.5 Force-push GitLab refusé

Cause : branche `main` protégée. **Solution** : déprotéger temporairement (Settings → Repository → Protected Branches), force-push, reprotéger.

16.6 Conflit après push CI

Cause : la CI a pushé un `[skip ci] Deploy image XXX` entre votre dernier pull et votre push. **Solution** : `git pull --rebase gitlab main` avant le push.

16.7 Aperçu Leaflet figé

Cause : exception silencieuse dans `calculateGridData()` (souvent une rotation extrême ou des coords invalides). **Solution** : ouvrir DevTools, regarder la console. La fonction est wrappée mais des erreurs peuvent passer.

16.8 KMZ exporté sans icônes

Cause : étiquettes générées en canvas mais ressources non embarquées (`cadoKmlResources.images` vide). **Solution** : vérifier que `generateKMZ()` appelle bien `JSZip.file('images/...')` pour chaque étiquette.

16.9 `Tailwind z-[10002]` n'a pas d'effet

Cause : Tailwind est précompilé statique, pas JIT. **Solution** : utiliser `style="z-index:10002"` inline.

Annexe A — Glossaire

Terme	Définition
Carroyage	Quadrillage de référence superposé à une carte
CADO	Système de carroyage maison (cellules à origine A1)
CFSI / DFCI	Carroyage Français de Sécurité Incendie / Défense de la Forêt Contre l'Incendie
UTM	Universal Transverse Mercator (projection cartographique)
MBTiles	Format SQLite pour bases de tuiles cartographiques (spec MapBox)
OPFS	Origin Private File System (stockage navigateur, hors quota localStorage)
TMS	Tile Map Service (convention de tuilage avec Y inversé)
Plus Code	Open Location Code, encodage compact de coordonnées
POI	Point Of Interest (marqueur utilisateur sur carte)
PWA	Progressive Web App
WKT	Well-Known Text (format texte de géométries)

Annexe B — Liens utiles

- IGN Géoplateforme : <https://geoservices.ign.fr/services-web-experts>

- Spec MBTiles : <https://github.com/mapbox/mbtiles-spec>
 - Service Worker MDN : https://developer.mozilla.org/fr/docs/Web/API/Service_Worker_API
 - Open Location Code : <https://github.com/google/open-location-code>
 - sql.js : <https://github.com/sql-js/sql.js>
 - Leaflet : <https://leafletjs.com/>
-

Documentation générée en collaboration avec Claude (Anthropic). Mise à jour : 2026-05-05.